МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №3

по дисциплине «Вычислительная математика»

Тема: Решение нелинейных уравнений. Метод Ньютона.

Студент гр. 4342 Ларионов В.А.

Преподаватель Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Вариант 10. Функция $f(x) = x^3 - 3 * x - 2 * e^{-x}$.

Изучить метод Ньютона для решения нелинейных уравнений. Найти корень заданного уравнения с заданной точностью, исследовать зависимость количества итераций от требуемой точности и оценить устойчивость метода к ошибкам в исходных данных.

Теоретический минимум.

В случае, когда известно хорошее начальное приближение решения уравнения $f(x) = 0$, эффективным методом повышения точности является метод Ньютона. Он состоит в построении итерационной последовательности $x_{n+1} = x_n - f(x_n)/f'(x_n)$, сходящейся к корню уравнения $f(x) = 0$.

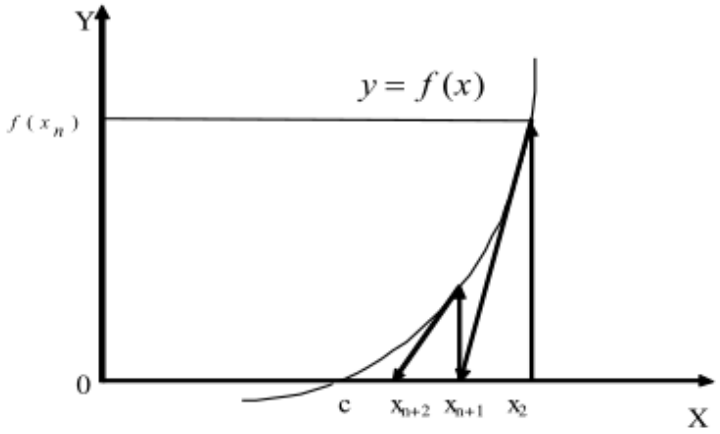

Рис.2

Метод Ньютона допускает простую геометрическую интерпретацию (рис. 2). Если через точку с координатами $(x_n; f(x_n))$ провести касательную, то абсцисса точки пересечения этой касательной с осью Ох будет очередным приближением x_{n+1} корня уравнения $f(x) = 0$. Для оценки погрешности n-го приближения корня предлагается пользоваться неравенством

$$|c - x_n| \leq \frac{M_2}{2m_1}|x_n - x_{n-1}|^2,$$

где M_2- наибольшее значение модуля второй производной $|f''(x)|$ на отрезке [a,b]; m_1- наименьшее значение модуля первой производной $|f'(x)|$ на

отрезке [a,b]. Таким образом, если $|x_n - x_{n-1}| < \varepsilon$, то $|c - x_n| \leq \frac{M_2 \varepsilon^2}{2m_1}$. Это означает, что при хорошем начальном приближении корня после каждой итерации число верных десятичных знаков в очередном приближении удваивается, т.е. процесс сходится очень быстро (имеет место квадратическая сходимость). Из указанного следует, что при необходимости нахождения корня с точностью итерационный процесс можно прекращать, когда

$$|x_n - x_{n-1}| < \varepsilon = \sqrt{2m_1 \varepsilon / M_2} \ . (3.1)$$

Рассмотрим один шаг итераций. Если на (n-1)-м шаге очередное приближение x_{n-1} не удовлетворяет условию окончания процесса, то вычисляются величины $f'(x_{n-1})$, $f'(x_{n-1})$ и следующие приближение корня $x_n = x_{n-1} - f(x_{n-1})/f'(x_{n-1})$. При выполнении условия (3.1) величина x_n принимается за приближенное значение корня с, вычисленное с точностью ε.

Выполнение работы.

1) С помощью графического веб-калькулятора desmos.com построим график (Рисунок 1) для определения отрезка $[a, b]$ близкого к $f(x) = 0$ на котором выполняется условие $f(a) * f(b) < 0$.

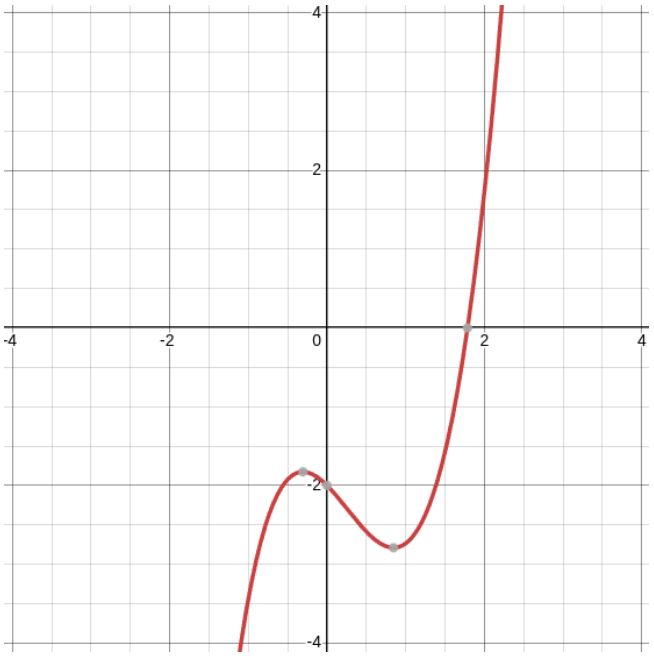

Рисунок 1 - график функции $f(x) = x^3 - 3 * x - 2 * e^{-x}$

На Рисунке 1 видно, что можно взять любой отрезок где $a \in (0.84472, 1.78546)$, а $b \in (1.78546, +\infty)$. Для простоты возьмем отрезок $[1.5, 2]$.

Для выполнения условия хорошего начального приближения, посчитал вторую производную и построил график (Рисунок 2). В промежутке $[1.5, 2]$, хорошим приближением будет $x_0 = 2$, так как выполняется условие $f''(x)f(x) > 0$.

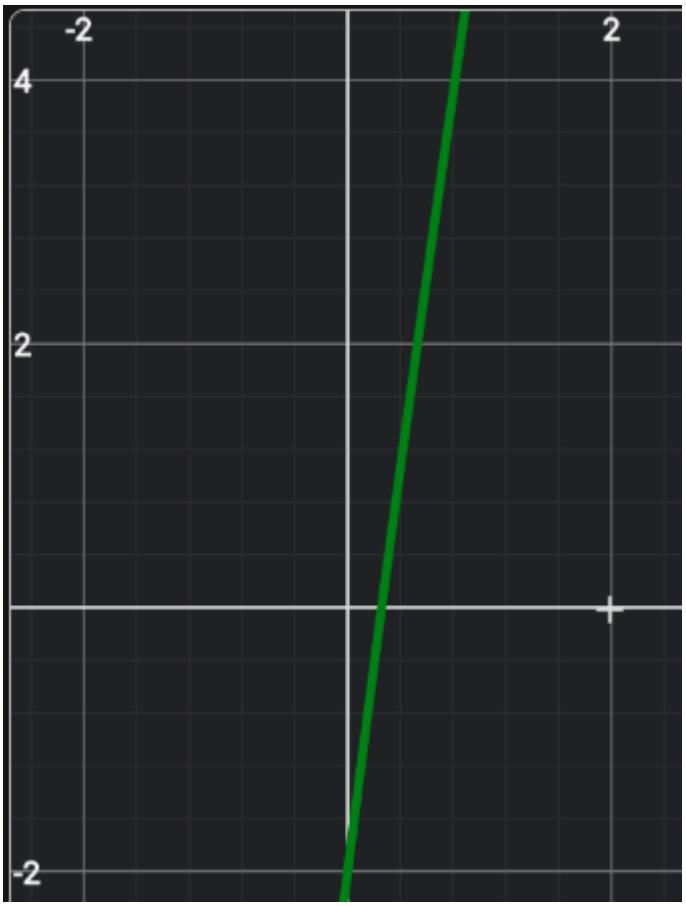

Рисунок 1 - график функции $f''(x) = 6 * x - 2 * e^{-x}$

2) Функция `extern double F(double x)` теперь возвращает значение `std::pow(x, 3) - 3 * x - 2 * std::pow(M_El, -x)` в соответствии с вариантом $f(x) = x^3 - 3 * x - 2 * e^{-x}$. Исходный код в Приложении А.
3) Создана функция находящаяся в файле main.cpp в Приложении А.
4) Проведены вычисления, построен график (Рисунок 2).

Рисунок 2 - “График” зависимости количества итераций от точности вычислений

Можно заметить, что на Рисунке 2 с ростом числа итераций точность вычислений растет экспоненциально и наоборот, чем меньшую допустимую погрешность (значение ε) мы задаем, тем меньше итераций нужно.

5) Для исследования чувствительности добавлены флаги компиляции ROUND и DELTA. При ROUND=1 DELTA задается автоматически значением 0.001, если нужно можно задать отдельно. Ниже в Таблице 1 приведены значения при разных дельта.

Таблица 1 - исследование чувствительности метода к ошибкам в исходных данных

Значение DELTA	ε	Значение возвращаемое функцией NEWTON	Количество итераций необходимых для заданной ε.
10^{-1}	10^{-1}	1.78943	2
	10^{-2}	1.78943	2
	10^{-3}	1.78943	2
	10^{-4}	1.78943	2
	10^{-5}	1.78943	2
	10^{-6}	1.78943	2
10^{-3}	10^{-1}	1.78598	2
	10^{-2}	1.7854	3
	10^{-3}	1.7854	3

	10^{-4}	1.7854	3
	10^{-5}	1.7854	3
	10^{-6}	1.7854	3
10^{-5}	10^{-1}	1.78603	2
	10^{-2}	1.78546	3
	10^{-3}	1.78546	3
	10^{-4}	1.78546	3
	10^{-5}	1.78546	3
	10^{-6}	1.78546	3
10^{-7}	10^{-1}	1.78603	2
	10^{-2}	1.78546	3
	10^{-3}	1.78546	3
	10^{-4}	1.78546	4
	10^{-5}	1.78546	4
	10^{-6}	1.78546	4

Выводы.

В ходе работы был изучен метод Ньютона. Для уравнения $f(x) = x^3 - 3 * x - 2 * e^{-x}$ корень был успешно найден на отрезке $[1.5, 2]$ с точностью от 0.1 до 0.000001. Построенный график показал логарифмическую зависимость числа итераций от точности: увеличение точности (уменьшение ε) требует ожидаемого роста количества вычислений, что соответствует теории. Исследование обусловленности показало, что метод хорд является устойчивым к малым ошибкам округления значений функции, однако при грубых

возмущениях (большая Delta) процесс может замедлиться или привести к потере точности нахождения корня.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <iostream>
#include <cmath>
#include "../../methods.hpp"

#ifndef LEFT
    // минимум для монотонности -- 0.84472
    #define LEFT 1.5L
#endif
#ifndef RIGHT
    // максимум -- 9223372036854775807 (максимальный double)
    #define RIGHT 2.L
#endif
#ifndef EPSILON
    int main() {
        int iterationsCount;
        for (double epsilon = 0.1; epsilon >= 0.000001; epsilon /= 10) {
            std::cout <<
                epsilon << "\t | " <<
                NEWTON(RIGHT, epsilon, iterationsCount) << "\t | " <<
                iterationsCount << '\n';
        }
        return EXIT_SUCCESS;
    }
#else
    int main() {
        int iterationsCount;
          std::cout << HORDA(LEFT, RIGHT, EPSILON, iterationsCount) << "\t | " <<
iterationsCount << '\n';
        return EXIT_SUCCESS;
    }
#endif
```

Файл methods.cpp

```
#include "methods.hpp"
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

#ifndef ROUND
    #define ROUND 0
#endif
#ifndef DELTA
    #define DELTA 0.001
#endif

#if !ROUND
double F(double x) {
    return std::pow(x, 3) - 3 * x - 2 * std::pow(M_El, -x);
}
double derivativeF(double x) {
    return 2 * std::pow(M_El, -x) + 3 * std::pow(x, 2) - 3;
}
#else
double F(double x) {
    return Round(std::pow(x, 3) - 3 * x - 2 * std::pow(M_El, -x), DELTA);
}
double derivativeF(double x) {
```

```
    return Round(2 * std::pow(M_El, -x) + 3 * std::pow(x, 2) - 3, DELTA);
}
#endif

double BISECT(double Left, double Right, double Eps, int &N)
{
    double E = fabs(Eps) * 2.0;
    double FLeft = F(Left);
    double FRight = F(Right);
    double X = (Left + Right) / 2.0;
    double Y;
    if (FLeft * FRight > 0.0)
    {
        puts("Неверное задание интервала\n");
        exit(1);
    }
    if (Eps <= 0.0)
    {
        puts("Неверное задание точности\n");
        exit(1);
    }
    N = 0;
    if (FLeft == 0.0)
        return Left;
    if (FRight == 0.0)
        return Right;
    while ((Right - Left) >= E)
    {
        X = 0.5 * (Right + Left);
        /* вычисление середины отрезка
         */
        Y = F(X);
        if (Y == 0.0) {
            if (derivativeF(X)) {
                return (X);
            }
            else {
                puts("Производная не равна нулю\n");
                exit(1);
            }
        }
        if (Y * FLeft < 0.0)
            Right = X;
        else
        {
            Left = X;
            FLeft = Y;
        }
        N++;
    };
    if (!derivativeF(X)) {
        puts("Производная не равна нулю\n");
        exit(1);
    }
    return (X);
}

double Round(double X, double Delta)
{
    if (Delta <= 1E-9)
    {
        puts("Неверное задание точности округления\n");
        exit(1);
```

```
    }
    if (X > 0.0)
        return (Delta * (long((X / Delta) + 0.5)));
    else
        return (Delta * (long((X / Delta) - 0.5)));
}

double ITER(double X0, double Eps, int &N)
{
    if (Eps <= 0.0)
    {
        puts("Неверное задание точности\n");
        exit(1);
    }
    double X1 = F(X0);
    double X2 = F(X1);
    N = 2;
    while ((X1 - X2) * (X1 - X2) > fabs((2 * X1 - X0 - X2) * Eps))
    {
        X0 = X1;
        X1 = X2;
        X2 = F(X1);
        N++;
    }
    return (X2);
}

#ifdef __NEWTON
double NEWTON(double X, double Eps, int &N)
{
    double Y, Y1, DX;
    N = 0;
    do
    {
        Y = F(X);
        if (Y == 0.0)
            return (X);
        Y1 = derivativeF(X);
        if (Y1 == 0.0)
        {
            puts("Производная обратилась в ноль\n");
            exit(1);
        }
        DX = Y / Y1;
        X = X - DX;
        N++;
    } while (fabs(DX) > Eps);
    return (X);

}
#endif

double HORDA(double Left, double Right, double Eps, int &N)
{
    double FLeft = F(Left);
    double FRight = F(Right);
    double X, Y;
    if (FLeft * FRight > 0.0)
    {
        puts("Неверное задание интервала\n");
        exit(1);
    }
    if (Eps <= 0.0)
```

```
    {
        puts("Неверное задание точности\n");
        exit(1);
    }
    N = 0;
    if (FLeft == 0.0)
        return Left;
    if (FRight == 0.0)
        return Right;
    do
    {
        X = Left - (Right - Left) * FLeft / (FRight - FLeft);
        Y = F(X);
        if (Y == 0.0)
            return (X);
        if (Y * FLeft < 0.0)
        {
            Right = X;
            FRight = Y;
        }
        else
        {
            Left = X;
            FLeft = Y;
        }
        N++;
    } while (fabs(Y) >= Eps);
    return (X);
}
```

Файл methods.hpp

```
extern double F(double);

#ifdef __NEWTON
extern double F1(double);
#endif

double Round(double X, double Delta);

double BISECT(double Left, double Right, double Eps, int &N);

double ITER(double X0, double Eps, int &N);

double HORDA(double Left, double Right, double Eps, int &N);

double NEWTON(double X, double Eps, int &N);
```